

TITLE :Write a Java program to demonstrate the use of an inner class, an anonymous class.

Problem Statement

Write a Java program to demonstrate the use of an inner class, an anonymous class. **Learning Objectives**

To implement inner classes and anonymous classes for improving encapsulation and event handling in Java.

Theory

An inner class is a class defined inside another class and has access to all members of the enclosing class, including private members. Inner classes help improve code organization and logical grouping of related classes. Anonymous classes are unnamed inner classes that are instantiated only once and are mainly used for implementing interfaces or abstract classes. They simplify code by eliminating the need for separate class definitions. These concepts are widely used in Java GUI programming and event handling

Conclusion

This application successfully demonstrates the concepts of inner classes and anonymous classes in Java. The member inner class enhances encapsulation and logical grouping by directly accessing the private members of the outer class. Additionally, the anonymous inner class provides a concise mechanism to instantiate and implement interfaces on the fly without defining separate standalone classes, reducing code verbosity and improving efficiency for event handling and single-use actions.

Exercise

1. Create a **Vehicle program** where an inner class displays vehicle details and anonymous class performs an action.

Code:

```
1  interface VehicleAction {
2      |   void performAction();
3      |
4  }
5  class Vehicle {
6      |   private String brand = "Toyota";
7      |   private String model = "Camry";
8      |
9      |   class VehicleDetails {
10         |       void display() {
11             |           System.out.println("Vehicle: " + brand + " " + model);
12         }
13     }
14     Run main | Debug main
15     public static void main(String[] args) {
16         |   Vehicle outerVehicle = new Vehicle();
17         |   Vehicle.VehicleDetails details = outerVehicle.new VehicleDetails();
18         |   details.display();
19         |
20         |   VehicleAction action = new VehicleAction() {
21             |       @Override
22             |       public void performAction() {
23                 |           System.out.println("Vehicle status: Engine started and driving.");
24             }
25         };
26         |   action.performAction();
27     }
```

Output:

```
Listening on 59947
User program running
Vehicle: Toyota Camry
Vehicle status: Engine started and driving.
User program finished
```

2. Develop a **Food Delivery Application** where an inner class handles order details and anonymous classes handle delivery status updates.

Code:

```
1 interface DeliveryStatus {
2     void updateStatus();
3 }
4 class FoodDeliveryApp {
5     private String orderId = "ORD9876";
6     private String item = "Pizza";
7     class OrderDetails {
8         void printOrder() {
9             System.out.println("Order ID: " + orderId + " | Item: " + item);
10        }
11    }
12    Run main | Debug main
13    public static void main(String[] args) {
14        FoodDeliveryApp app = new FoodDeliveryApp();
15
16        FoodDeliveryApp.OrderDetails order = app.new OrderDetails();
17        order.printOrder();
18        DeliveryStatus status = new DeliveryStatus() {
19            @Override
20            public void updateStatus() {
21                System.out.println("Delivery Status: Out for Delivery.");
22            }
23        };
24        status.updateStatus();
25    }
26 }
```

Output:

```
Listening on 49312
User program running
Order ID: ORD9876 | Item: Pizza
Delivery Status: Out for Delivery.
User program finished
```